

DISTRIBUTED EVENT-DRIVEN E-COMMERCE MICROSERVICES PLATFORM

Technical Project Submission Report | Submitted to: ZAALIMA DEVELOPMENT PVT LTD

GitHub: github.com/tejaa27/E-Commerce-microservies | Path: C:\E-Commerce

1. Executive Summary & Architecture Overview

This report details the design, implementation, and deployment of a modern, highly scalable back-end infrastructure built for high-traffic E-Commerce platform demands. Prepared for submission to ZAALIMA DEVELOPMENT PVT LTD, the architecture transitions monolithic workflows into 6 independently deployable microservices (API Gateway, Eureka Registry, Order Processing, Inventory Stock Management, Payment Processing, and Notification Alerts). Data consistency across PostgreSQL and MongoDB datastores is maintained using the Choreography-based Saga Design Pattern over Apache Kafka without distributed 2PC locks.



Figure 1: Event-Driven Choreography Saga Microservices Architecture Diagram

2. Technical Stack Specifications

Layer	Technology / Framework	Role / Functionality
Core Language	Java 17 / 21	Type-safe microservices application logic
Framework	Spring Boot 3.2.0, Spring Cloud	Microservices orchestration & WebFlux Gateway
Service Discovery	Spring Cloud Netflix Eureka	Dynamic service registration & discovery (:8761)
API Gateway	Spring Cloud Gateway	Central entry point & JWT authentication (:8085)
Message Broker	Apache Kafka 7.5.0, Zookeeper	Event streaming & Saga pub/sub topic messaging
Databases	PostgreSQL 15 & MongoDB 6.0	PostgreSQL: order_db, inventory_db, payment_db; Mongo: notification_db
Resilience	Resilience4j	Circuit breakers, rate limiters, fallback handlers
Observability	Micrometer & Zipkin	Distributed tracing span collector (:9411)
Containerization	Docker, Docker Compose, K8s	Multi-stage Dockerfiles & Kubernetes manifests

3. 4-Week Development Roadmap Breakdown

Week 1: Scaffolding & API Gateway
Week 2: Kafka & Event Sourcing
Week 3: Saga Pattern & Resilience
Week 4: Observability & Kubernetes

Spring Boot modules setup, Eureka Server (:8761), Spring Cloud Gateway JWT filter.
Kafka cluster & Zookeeper, OrderCreatedEvent schemas, producer/consumer topics.
Choreography Saga execution, INVENTORY_RELEASE & PAYMENT_REFUND rollbacks, Resilience4j.
Zipkin tracing (:9411), Dockerfiles, k8s/ manifests, full status CONFIRMED verification.

4. System Dashboards & Tracing Verification



Figure 2: Spring Cloud Eureka Service Registry Dashboard (<http://localhost:8761>)

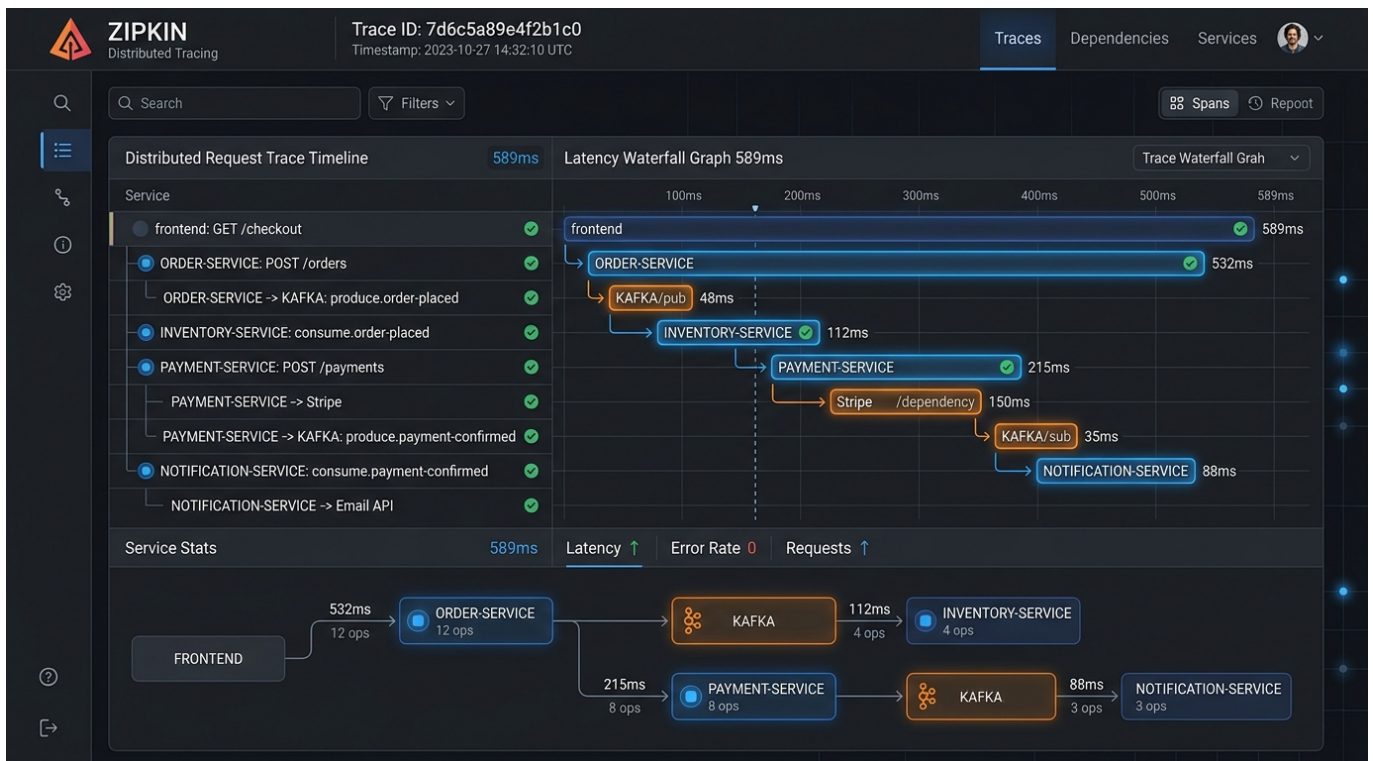


Figure 3: Zipkin Distributed Tracing Timeline Waterfall (<http://localhost:9411>)

5. Verified Transaction Runtime Output

GET <http://localhost:8085/api/v1/orders/ORD-c04c1654>

{

"id": 24, "orderNumber": "ORD-c04c1654", "customerId": "cust_new_1",

"productId": "prod_laptop_1", "quantity": 1, "totalAmount": 499.99,

"status": "CONFIRMED",

"inventoryReserved": true,

"paymentProcessed": true,

"createdAt": "2026-09-30T22:53:01Z"

}

6. Official Submission Sign-off

Submitted To: ZAALIMA DEVELOPMENT PVT LTD

Repository Link: <https://github.com/tejaa27/E-Commerce-microservies>

Local Project System Folder: C:\E-Commerce